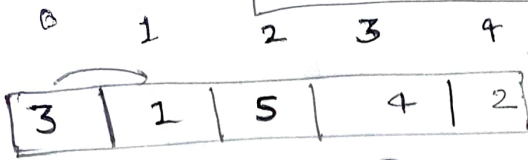
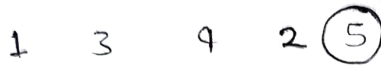
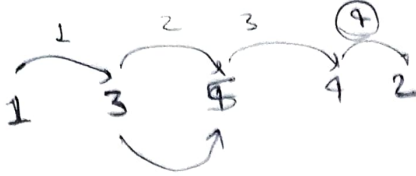


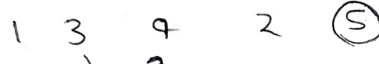
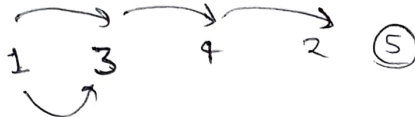
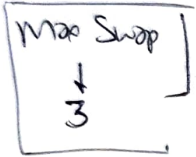
BUBBLE SORT



1st Pass



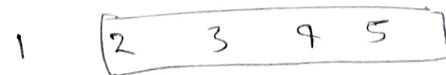
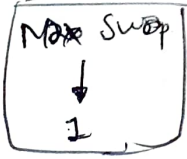
2nd Pass



3rd Pass



4th Pass



5th Pass

TC Best (Sorted) = $O(n)$
Worst = $O(n^2)$

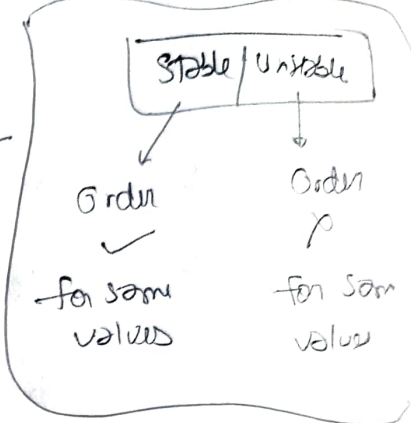
Stable

Comparing adjacent element

Swap

X

X



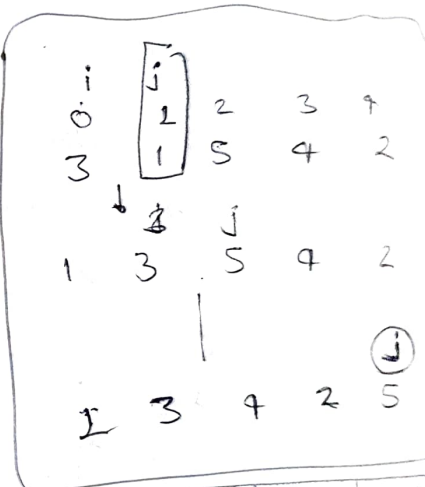
X

X

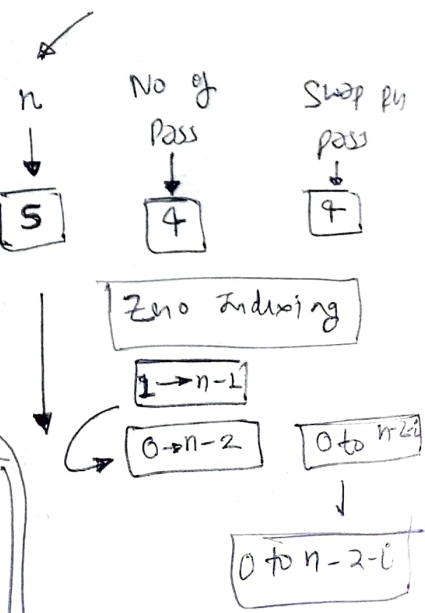
✓

X

✓



No of element in an



SELECTION SORT

Select an element & put it at its correct position.

0 1 2 3 4
4 5 1 2 3

Selecting largest

4 3 1 2 5

2 3 1 4 5

2 1 3 4 5

1 2 3 4 5

0 1 2 3 4

4 5 1 2 3

Selecting Smallest

1 5 4 2 3

1 2 4 5 3

1 2 3 5 4

1 2 3 4 5

Size of arr = $n = 5$

Comparisons = $n - 1$

Total Comparisons = $0 + 1 + 2 + \dots + (n-1) = \frac{(n-1) \cdot n}{2}$
 $= \frac{n^2 - n}{2} = \frac{n^2}{2}$

Not Stable

• Perform well on small lists.

Worst Case $O(n^2)$
 Best Case $O(n^2)$

static void selectionSort(int[] nums)

for (int i = 0; i <= nums.length - 2; i++)

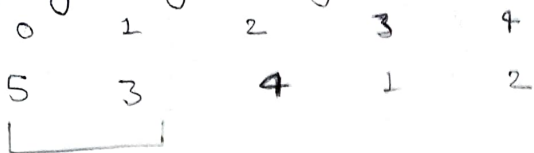
{ int last = nums.length - 1 - i;

int maxIndex = getMaxIndex(nums, 0, last);

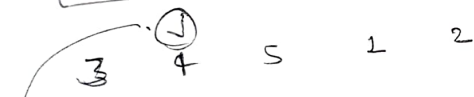
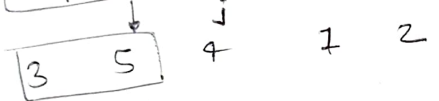
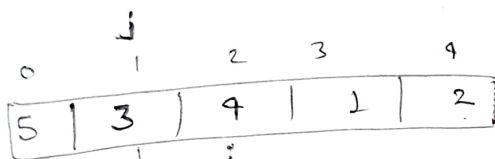
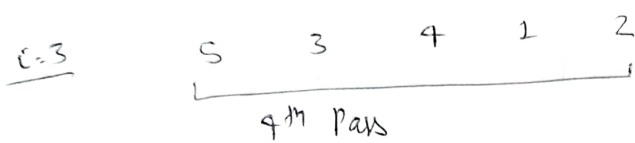
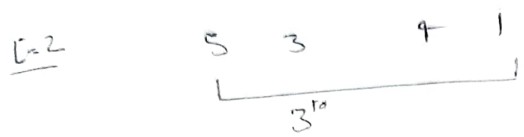
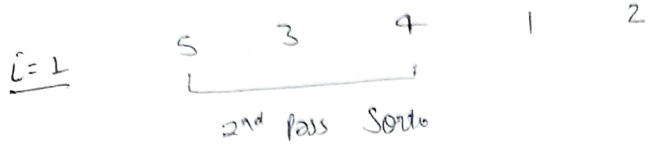
swap(nums, maxIndex, last);

INSERTION SORT

Sorting Array Partially



$i=0$
1st Pass Sorts



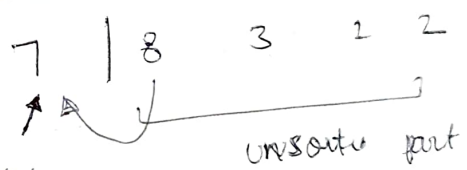
Sort till index	Pass No	$i \leq n-2$	$j > 0$
1	$\leftarrow 1$	0	1
2	$\leftarrow 2$	1	2
3	$\leftarrow 3$	2	
4	$\leftarrow 4$	3	

When element '1' is not smaller than element $j-1$, break;

Why Insertion Sort?

- ① Stable
- ② Steps gets reduced
- ③ Used for smaller value of N
- ④ Works good when array is partially sorted

For every index you are at put that index element of the correct index on LHS.



Sorted

